Getting Started with Python

Chapter 2 | L1 Socle Commun Sciences de la Matière
Université Frères Mentouri Constantine 1
Dr. Farida Larit | Module coordinator
Student course notes | Two 90-minute lectures

Learning outcomes

- Create, save and run a Python 3 script in IDLE.
- Use int, float, str and bool appropriately.
- Convert input and evaluate arithmetic, comparison and logical expressions.
- Translate a sequential scientific algorithm and check its units.

Prerequisites

Chapter 1: input–processing–output, variables, arithmetic and tracing.

Session	Focus
1	IDLE, variables, constants, types and assignment
2	Operators, input/output and scientific calculations

How to study: read the explanation, predict the result, trace the example, then attempt the checkpoint. Write your reasoning before consulting the teacher correction.

2.1 Recall: design before execution

An algorithm describes the method. Python expresses that method with precise syntax. Begin with the previous density algorithm: identify the two inputs, the division and the labelled output. For Chapter 2 examples, numeric input and valid physical values are preconditions. Chapter 3 introduces checks for invalid values.

Worked example

```
mass = 10.0
volume = 2.0
density = mass / volume
print("Density =", density, "g/mL")
```

Python executes these statements in order. Decimal numbers use a point. Text labels are quoted. A variable name is not quoted when its stored value is needed.

Check your understanding

What would print("density") display?

Your explanation: _____

_____

2.2 Working in IDLE

Use an installed Python 3 environment. IDLE has a Shell for immediate experiments and an Editor for saved scripts. In the Editor, create a new file, save it with a .py extension, then use Run Module (F5). Output and input prompts appear in the Shell. Do not copy the >>> Shell prompt into a script.

Worked example

```
print("My first scientific program")
print(2 + 3)
# Save as first_program.py
```

For the first live demonstration, point to the Editor before running and then point to the Shell. Save the file in a course folder. Avoid naming files math.py or random.py because such names can hide standard modules.

Check your understanding

Why might a script run yesterday’s code?

Your explanation: _____

_____

2.3 Names, values and constants

A Python variable is a name bound to an object. For beginner numeric examples, think of a labelled value that can be updated. Names are case-sensitive and may contain letters, digits and underscores, but cannot start with a digit. Keywords such as if cannot be used as names. Uppercase constants are a convention, not an enforced restriction.

Worked example

```
mass_g = 12.5
number_of_samples = 4
GRAVITY = 9.81
mass_g = mass_g + 0.5
print(mass_g)
```

The final value is 13.0. Prefer names that carry scientific meaning and units. Avoid reusing built-in names such as input, print, sum and list. A constant name can technically be rebound, so discipline matters.

Check your understanding

Which names are valid: 2mass, mass_g, total-mass, Mass?

Your explanation: _____

_____

2.4 Fundamental types and conversion

int represents integers, float represents approximate real-number values, str represents text and bool represents True or False. Python has no separate character type: a one-character value is a str. A value’s type affects the operation performed. float values are not exact representations of every decimal fraction.

Worked example

```
count = 3
mass = 3.0
label = "3"
valid = True
print(type(count), type(mass))
print(label + label)
print(int(label) + int(label))
```

The numeric types are int and float. Concatenating the two strings produces 33; converting both to integers produces 6. int(3.9) gives 3, which truncates toward zero. int("3.9") fails.

Check your understanding

Predict 2 + 3.0 and "2" + "3".

Your explanation: _____
_____

2.5 Arithmetic and precedence

Use +, -, *, /, //, %, and **. For ordinary int and float operands, / produces a float. // is floor division, which rounds the quotient downward, including for negative values. % gives the corresponding remainder. Parentheses should make scientific formulas unambiguous. Exponentiation is **, not ^.

Worked example

```
print(10 / 3)
print(10 // 3, 10 % 3)
print(-7 // 2, -7 % 2)
print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3)
```

Outputs are approximately 3.3333, then 3 and 1, then -4 and 1, then 14, 20 and 8. For kinetic energy use 0.5 * mass * speed**2. Multiplication signs cannot be omitted.

Check your understanding

Translate (a+b)/(2c) into Python.

Your explanation: _____

_____

2.6 Comparison, logic and assignment

= assigns a value; == compares values. Comparisons include !=, <, <=, > and >=. Combine Boolean conditions using and, or and not. Assignment first evaluates the right-hand side. For numeric values x += 1 updates x by one. Simultaneous assignment can swap values.

Worked example

```
x = 5
y = 2
print(x > 0 and y > 0)
x, y = y, x
x += 1
print(x, y)
```

The first output is True. The swap gives x = 2 and y = 5; the update gives x = 3. Write 0 <= x <= 10 to test a closed interval.

Check your understanding

For x = 0, compare x > 0 and x >= 0.

Your explanation: _____

_____

2.7 Reading and displaying scientific data

input() returns a string. Convert it using float() for measurements or int() for whole-number counts. Prompts should specify the unit. print() can combine text and values. Formatting controls the display, not the accuracy of the measurement. Invalid numeric text raises ValueError; exception handling is outside the core of this chapter.

Worked example

```
mass = float(input("Mass (g): "))
volume = float(input("Volume (mL), > 0: "))
density = mass / volume
print(f"Density = {density:.3f} g/mL")
```

For inputs 10 and 2, the output is Density = 5.000 g/mL. The .3f format gives three digits after the decimal point. This example assumes a positive volume.

Check your understanding

What happens if the conversions are removed?

Your explanation: _____

_____

2.8 Scientific calculations and debugging

Check the formula, units, allowed inputs and expected answer before typing. Use comments to explain assumptions. Distinguish a syntax error from a runtime error and a logical error. A program can run and still calculate the wrong result. Mathematical library functions will be consolidated in Chapter 5.

Worked example

```
mass_kg = float(input("Mass (kg): "))
speed_ms = float(input("Speed (m/s): "))
energy_j = 0.5 * mass_kg * speed_ms**2
print("Kinetic energy =", energy_j, "J")
```

For 2 kg and 3 m/s the result is 9 J. For a mass of 500 g, first convert to 0.5 kg. Using grams directly would introduce a factor of 1000.

Check your understanding

Correct energy = 0.5 * mass * speed * 2.

Your explanation: _____

_____

Chapter synthesis and revision

- Can I create, save and run a Python 3 script in IDLE?
- Can I use int, float, str and bool appropriately?
- Can I convert input and evaluate arithmetic, comparison and logical expressions?
- Can I translate a sequential scientific algorithm and check its units?

English	French
input / output	entrée / sortie
assignment	affectation
condition / branch	condition / branche
loop / iteration	boucle / itération
trace / test	trace / test
pseudocode / flowchart	pseudo-code / organigramme

Reference reading

Python Software Foundation. Python 3 Tutorial: An Informal Introduction to Python. https://docs.python.org/3/tutorial/introduction.html

Python Software Foundation. More Control Flow Tools. https://docs.python.org/3/tutorial/controlflow.html

Python Software Foundation. math: Mathematical functions. https://docs.python.org/3/library/math.html

Python Software Foundation. IDLE: Python editor and shell. https://docs.python.org/3/library/idle.html